

第七章 查找

一. 基本概念.

1. 查找：在数据集合中寻找满足某种条件的数据元素的过程
2. 查找表：用于查找的数据集合
3. 关键字：数据元素中唯一标识该元素的某个数据项的值 (基于关键字查找，结果唯一)
4. 平均查找长度 (ASL)：所有查找过程中进行关键字的比较次数的平均值
衡量查找效率的最主要的指标

静态查找表：只作查找操作的查找表

动态查找表: ①表结构是在查找过程中动态生成
②查找时插入或删除数据元素

- 二叉排序树
 - 平衡二叉树
 - B树
 - 散列表

二. 线性结构

1. 顺序查找

(1) 一般线性表

```
typedef struct {
    ElemType *elem;
    int TableLen;
}
```

YSSTable;

```
int Search_Seq (SSTable ST, ElemType key) {
```

ST.elem[0] = key; //哨兵, 使得循环不必判断数组是否会越界
for (int i = ST.TableLen; ST.elem[i] != key; --i); //从后往前
return i;

```
return i;
```

(2) 有序线性表 (优化)

$$\textcircled{1} \text{ ASL}_{\text{取}} = \frac{n}{\sum_{j=1}^n q_j} (l_j - 1) = \frac{1+2+\dots+n+n}{n+1}$$

$$\downarrow$$

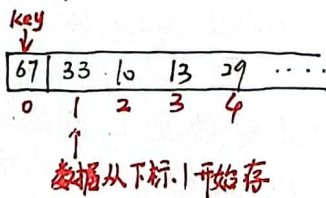
$$\text{取} = \frac{n}{2} + \frac{n}{n+1}$$

到达第 j 个未知结点
的概率，相等下为 $\frac{1}{m}$

② ASL成功 = $\frac{n+1}{2}$

③失败结点、的查找长度 = 父结点^(50, 60)层数^(60, ∞) $\text{length}(25) = 3$

④成功结点, 的查找长度 = 自身层数 $\text{length}(p_0) = 4$

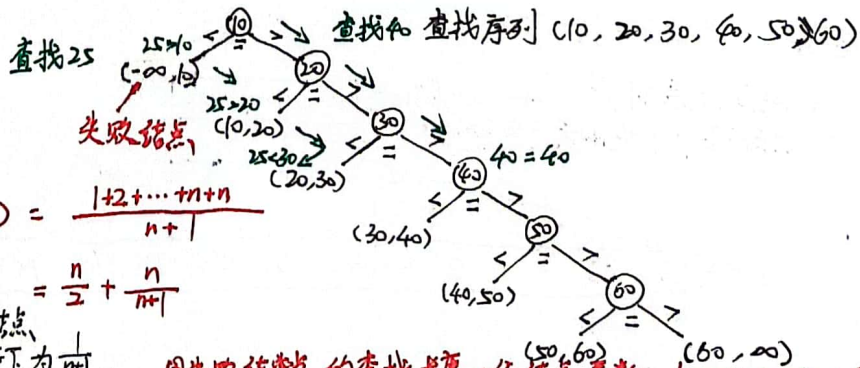


$$\textcircled{1} \text{ ASL成功} = \sum_{i=1}^{\infty} P_i (n-i+1)$$

$$= \frac{n+1}{2} \quad (\text{每个元素查找概率相等, } P_i = \frac{1}{n})$$

② ASL失败 $= n + 1$

③ 时间复杂度 $O(n)$



2. 折半查找 (二分查找) 仅适用于有序的顺序表

(1) 基本思想

有序表: {7, 10, 13, 16, 19, 29, 32, 33, 37, 41, 43}

```
int Binary_Search (SSTable L, ElemType key) {
```

```
    int low = 0, high = L.TableLen - 1, mid;
```

```
    while (low <= high) {
```

```
        mid = (low + high) / 2;
```

// 取中间位置 (取整方向保持一致)

```
        if (L.elem[mid] == key)
```

```
            return mid;
```

```
        else if (L.elem[mid] > key) // 左边查找
```

```
            high = mid - 1;
```

```
        else
```

```
            low = mid + 1;
```

// 右边查找

```
    }
```

```
    return -1;
```

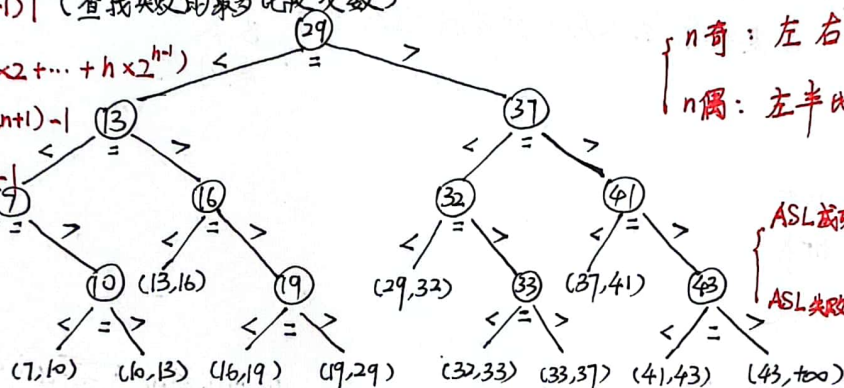
```
}
```

(2). 判定树: 折半查找的过程

① 树高 $h = \lceil \log_2(n+1) \rceil$ (查找失败的最多比较次数)

② ASL成功 = $\frac{1}{n}(1 \times 1 + 2 \times 2 + \dots + h \times 2^{h-1})$
 $= \frac{n+1}{n} \log_2(n+1) - 1$
 $\approx \log_2(n+1) - 1$

③ 时间复杂度 $O(\log_2 n)$
 $(-\infty, -1)$



n 奇: 左右两部分元素个数相等
 n 偶: 左半比右半少一个元素

ASL成功 = $\frac{1 \times 1 + 2 \times 2 + 3 \times 4 + 4 \times 4}{11} = 3$
 ASL失败 = $\frac{3 \times 4 + 4 \times 8}{12} = \frac{11}{3}$

① 查找成功的查找长度为从根结点到目的结点的路径上的结点数

② 查找失败的查找长度为从根结点到对应失败结点的父结点的路径上的结点数

③ 若有序序列有 n 个元素, 判定树有 n 个非叶结点和 $n+1$ 个叶结点

④ 判定树是一棵二叉平衡树

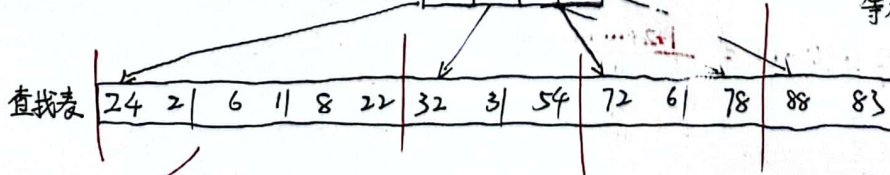
3. 分块查找 (索引顺序查找)

(1) 基本思想: 关键码 {88, 24, 72, 61, 21, 6, 32, 11, 8, 31, 22, 83, 78, 54}

① 块间元素有序 (保存每个分块的最大关键字)

索引表

24	54	78	88
1	7	10	13



将长度为 n 的查找表均匀地分为 b 块, 每块有 s 个记录, 等概率下, 若均采用顺序查找

ASL = $L_s + L_b$
 $= \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2sm}{2s}$
 若 $S = \sqrt{n}$, $ASL_{min} = \sqrt{n+1}$

② 块内元素无序

目标记录不在索引表中, 折半查找最终停在 $low > high$, 在 low 所指分块中查找

(2) 查找过程: ① 在索引表找待查记录所在块 (顺序/折半)
 ② 在块内顺序查找

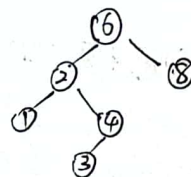
三. 树形结构

- 二叉排序树
- 二叉平衡树
- 红黑树
- B树, B⁺树

1. 二叉排序树 BST

- (1) 定义:
- ① 若左子树非空, 则左子树上所有结点的值均小于根结点的值;
 - ② 若右子树非空, 则右子树上所有结点的值均大于根结点的值;
 - ③ 左、右子树也分别是一棵二叉排序树。

对BST进行中序遍历, 可以得到一个递增的有序序列



123468

(2) BST的查找

① 非递归

```

BSTNode *BST_Search (BiTree T, ElemType key) {
    while (T != NULL && key != T->data) {
        if (key < T->data) T = T->lchild; // 小于, 在左子树上查找
        else T = T->rchild; // 大于, 在右子树上查找
    }
    return T;
}

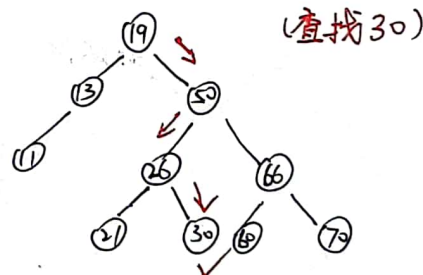
```

② 递归

```

BSTNode *BST_Search (BiTree T, ElemType key) {
    if (T == NULL) return NULL;
    if (key == T->key) return T;
    else if (key < T->key)
        return BST_Search (T->lchild, key);
    else
        return BST_Search (T->rchild, key);
}

```

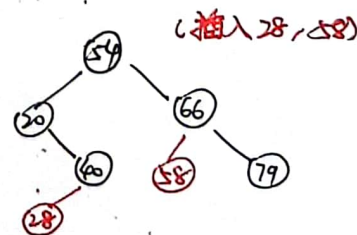


(3) BST的插入 时间复杂度 $O(\log_2 n)$

```

int BST_Insert (BiTree &T, keyType k) {
    if (T == NULL) { // 原树为空, 新插入的记录为根结点
        T = (BiTree) malloc (sizeof (BSTNode));
        T->data = k;
        T->lchild = T->rchild = NULL;
        return 1;
    }
    else if (k == T->data) // 树中存在相同关键字的结点, 失败
        return 0;
    else if (k < T->data) // 插入T的左子树
        return BST_Insert (T->lchild, k);
    else // 插入T的右子树
        return BST_Insert (T->rchild, k);
}

```

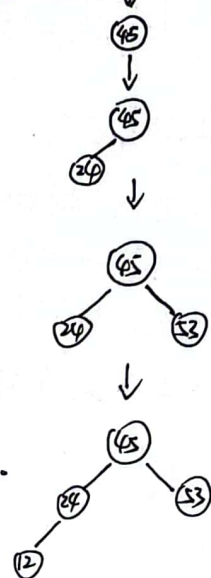


(4) BST 的构造

```
void Creat-BST(BiTree &T, keyType str[], int n) {
    T = NULL;
    int i = 0;
    while (i < n) {
        // 依次将每个关键字插入 BST
        BST-Insert(T, str[i]);
        i++;
    }
}
```

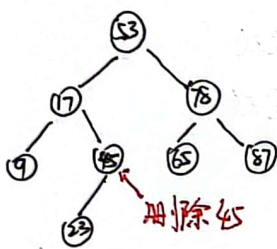
str = {45, 24, 53, 45, 12, 24}

构造过程: $\emptyset$ (空树)

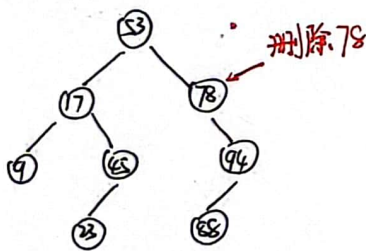
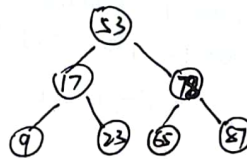


(5) BST 的删除 时间复杂度 $O(\log_2 n)$

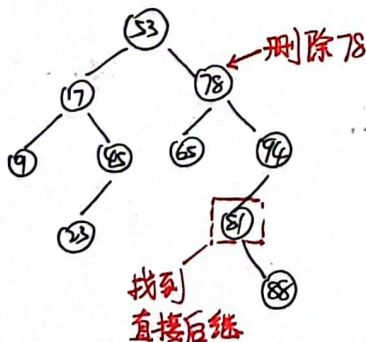
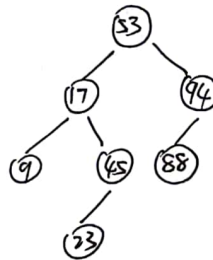
- ① 若被删除结点 Z 是叶结点, 直接删;
- ② 若 Z 只有一棵左/右子树, 子替父;
- ③ 若 Z 有左右两棵子树, Z 的直接后继/前驱替代, 并删该后继/前驱。



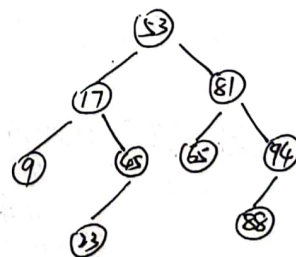
右子树空,
用左子女补



左子树空,
用右子女补



左、右子树均不空,
在右子树找中序
第一个子女填补
替换为删除 81



(6) BST 的查找效率

- ① 若是平衡二叉树, 平均查找长度为 $O(\log_2 n)$.
 - ② 若是一个只有左/右孩子的单支树, 平均查找长度为 $O(n)$.
- 有序表是静态查找表时, 采用顺序表作为存储结构, 使用二分查找
有序表是动态查找表时, 采用二叉排序树作为逻辑结构

2. 平衡二叉树 AVL

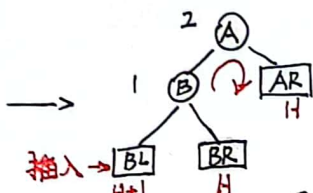
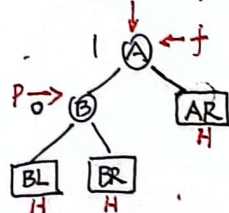
- (1) 定义: ① 规定二叉排序树的左、右子树的高度之差的绝对值不超过 1
② 高度差为该结点的平衡因子 $(-1, 0, 1)$

(2) AVL 的插入

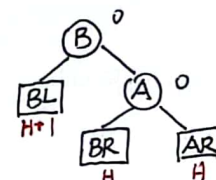
每次调整最小不平衡子树

① LL 平衡旋转 (右单旋转)

$gf \rightarrow lchild/rchild$



LL 旋转



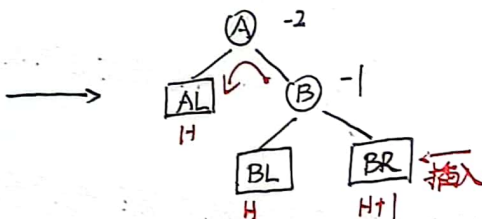
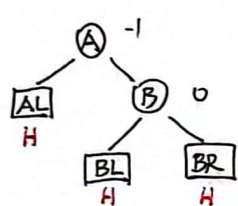
保证: $BL < B < BR < A < AR$

① $f \rightarrow lchild = P \rightarrow rchild$;

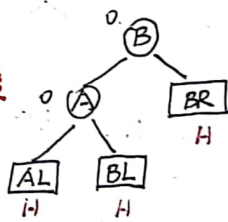
② $P \rightarrow rchild = f$;

③ $gf - lchild/rchild = P$;

② RR 平衡旋转 (左单旋转)



RR 旋转



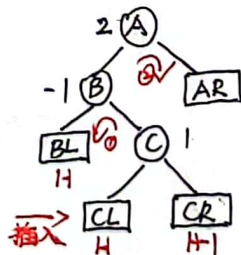
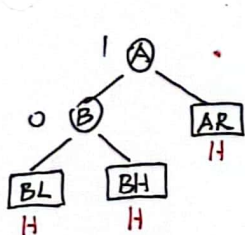
保证: $AL < A < BL < B < BR$

① $f \rightarrow rchild = P \rightarrow lchild$;

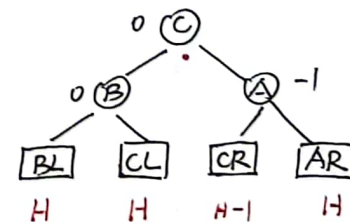
② $P \rightarrow lchild = f$;

③ $gf \rightarrow lchild/rchild = P$;

③ LR 平衡旋转 (先左后右双旋转)

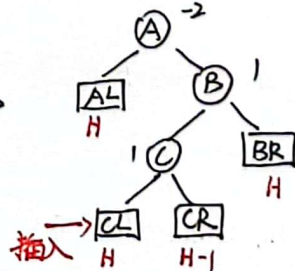
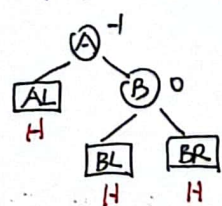


LR 旋转

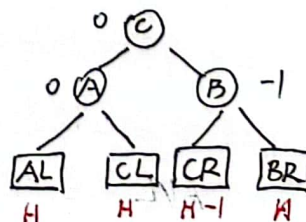


保证: $BL < B < CL < C < CR < A < AR$

④ RL 平衡旋转 (先右后左双旋转)

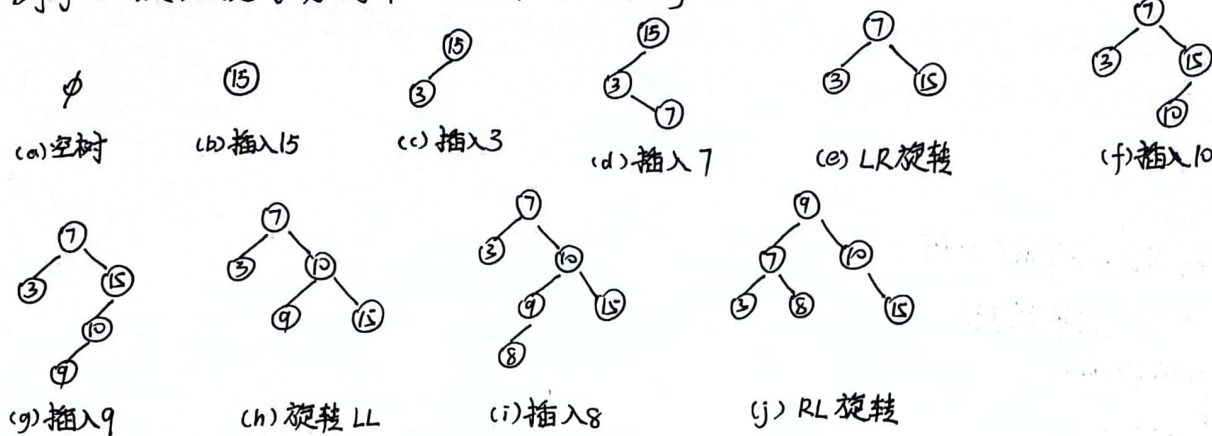


RL 旋转



保证: $AL < A < CL < C < CR < B < BR$

例子：以关键字序列 {15, 3, 7, 10, 9, 8}

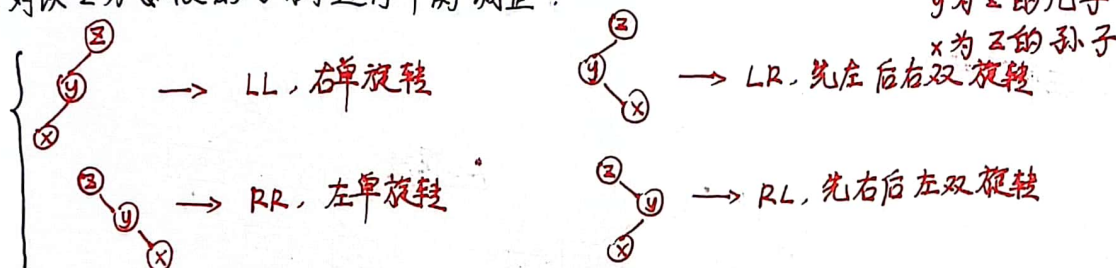


(3) AVL 的删除

① 用二叉排序树的方法对结点 w 执行删除操作。

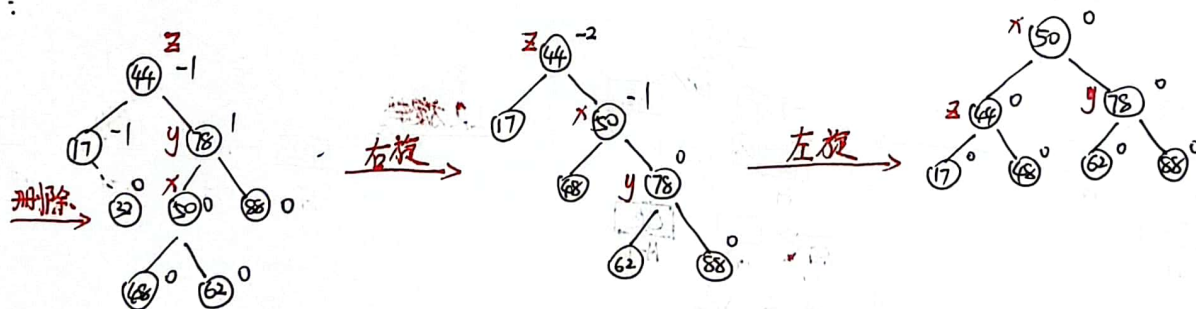
② 若导致了不平衡，从 w 向上找到第一个不平衡的结点 z 。

③ 对以 z 为根的子树进行平衡调整：



注：若调整后子树的高度减 1，则可能需要对 z 的祖先结点进行平衡调整，甚至回溯到根结点（导致树高减 1）。

例子：



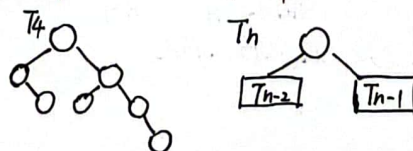
(4) AVL 的查找

① 过程与二叉排序树的相同

② 含有 n 个结点的 AVL 的最大深度为 $O(\log_2 n)$ ，平均查找效率为 $O(\log_2 n)$

③ 深度为 h 的 AVL 中含有的最多结点数为 $2^h - 1$ （满二叉树）

④ 深度为 h 的 AVL 中含有的最少结点数为 $n_h = n_{h-2} + n_{h-1} + 1$ ($n_0 = 0, n_1 = 1, n_2 = 2$)

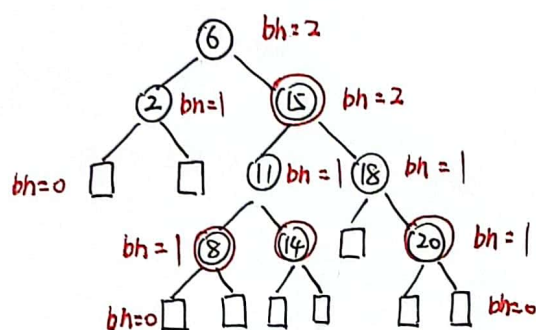


3. 红黑树

(1) 定义：左根右，根叶黑，不红红，黑路同。

满足如下性质的二叉排序树：

- ① 每个结点，是红色或黑色。
- ② 根结点，黑色。
- ③ 叶结点（虚构的外部结点、NULL 结点）都是黑色。
- ④ 不存在两个相邻的红结点（即红结点的父和孩子结点，均是黑色）。
- ⑤ 对每个结点，从该结点，到任意一个叶结点的简单路径上，所含黑结点的数量相同。



□ 空结点，（引入 $n+1$ 个，保证每个内部结点的左、右孩子非空）
○ 黑结点
○ 红结点

结点的黑高(bh)：从某结点出发（不含该结点）到达一个叶结点的任意一个简单路径上的黑结点总数

- 结论：
- ① 从根到叶结点，的最长路径不大于最短路径的 2 倍
 - ② 有 n 个内部结点的红黑树的高度 $h \leq 2\log_2(n+1)$
 - ③ 查找时间复杂度 $O(\log_2 n)$

(2) 红黑树的插入

· 先查找，确定插入位置（原理同二叉排序树），插入新结点

· 新结点是根 —— 染为黑色

· 新结点，非根 —— 染为红色

· 若插入新结点后依然满足红黑树定义，则插入结束

· 若插入新结点后不满足红黑树定义，需要调整：——→ 找结点的叔

· 黑叔：旋转 + 染色

· LL 型：右单旋，父换爷 + 染色

· RR 型：左单旋，父换爷 + 染色

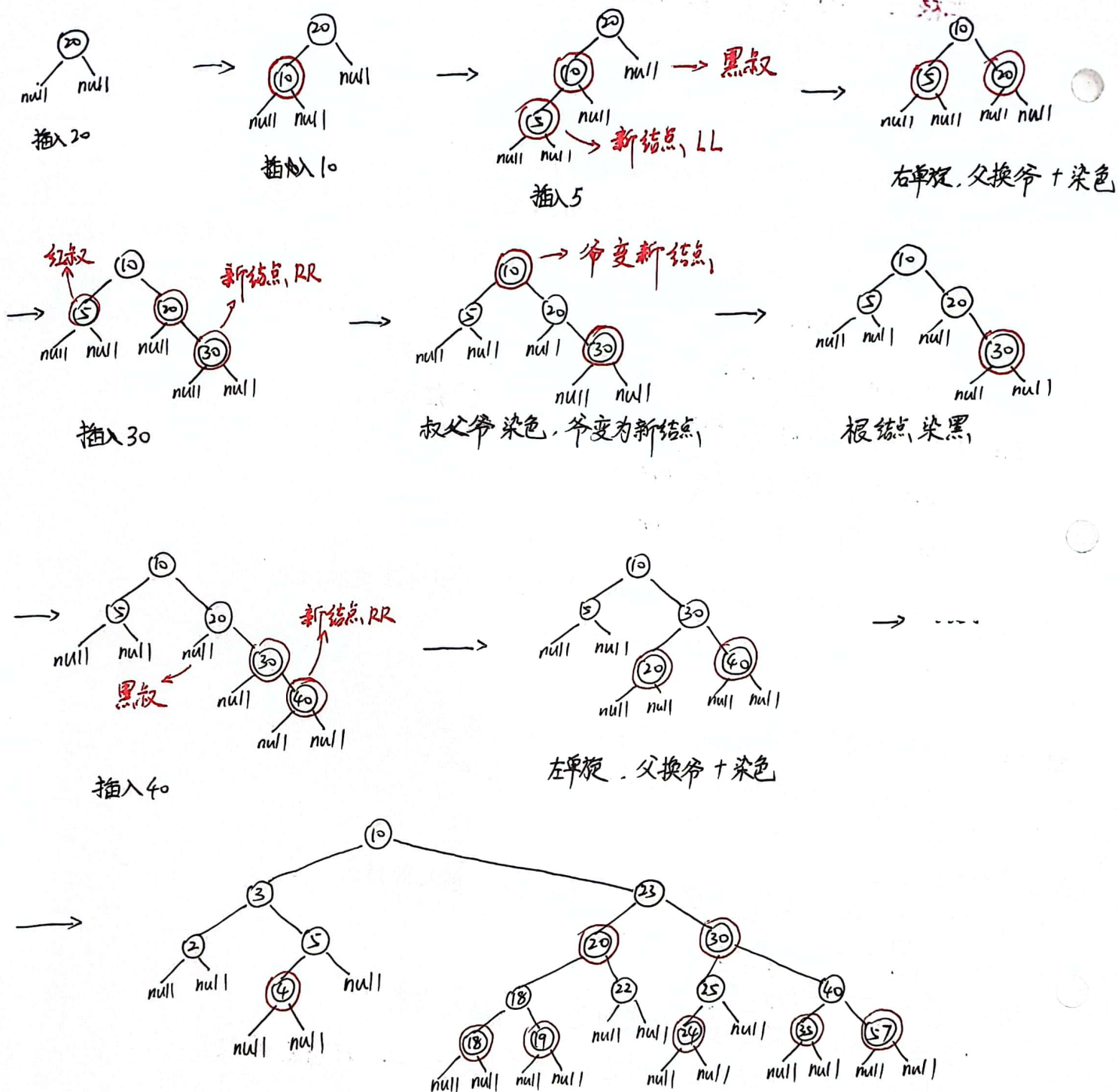
· LR 型：左、右双旋，儿换爷 + 染色

· RL 型：右、左双旋，儿换爷 + 染色

· 红叔：染色 + 变新

· 叔父爷染色，爷变为新结点

例子: 插入 {20, 10, 5, 30, 40, 57, 3, 2, 4, 35, 25, 18, 22, 23, 24, 19, 189}



(3) 红黑树的删除

① 时间复杂度 $O(\log_2 n)$.

② 处理方式与“二叉排序树的删除”一样。

③ 删除结点后, 若破坏了“红黑树的特性”, 需要调整结点的颜色、位置。

4. B树

(1) 定义：又称多路平衡查找树。

m 阶B树是所有结点，的平衡因子均等于0。

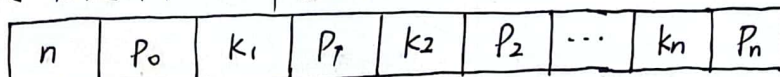
① 树中每个结点，至多有 m 棵子树，即至多含有 $m-1$ 个关键字。

② 若根结点，不是叶结点，则至少有两棵子树。

③ 除根结点外的所有非叶结点，至少有 $\lceil m/2 \rceil$ 棵子树，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。

④ 所有叶结点都出现在同一层次上，且不带信息。

⑤ 所有非叶结点的结构为

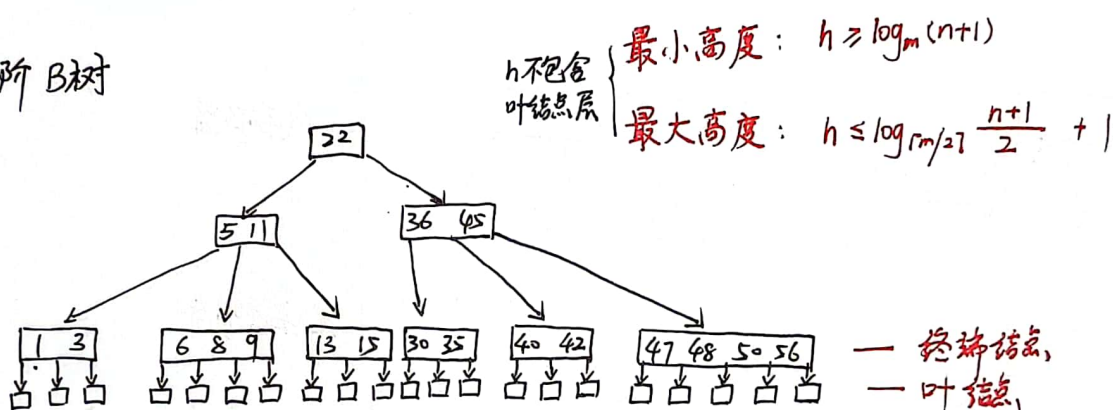


$K_i (i=1, 2, \dots, n)$ 为关键字

$P_i (i=0, 1, 2, \dots, n)$ 指向子树

P_{i-1} 的关键字 $< K_i < P_i$ 的关键字

例子：5阶B树



m 阶B树的核心特性：

① 根结点的子树数 $\in [2, m]$ ；关键字数 $\in [1, m-1]$

其他结点的子树数 $\in [\lceil m/2 \rceil, m]$ ；关键字数 $\in [\lceil m/2 \rceil - 1, m-1]$ 。

② 对任一结点，其所有子树高度相同。

③ 关键字的值：子树0 < 关键字1 < 子树1 < 关键字2 < 子树2 < ...

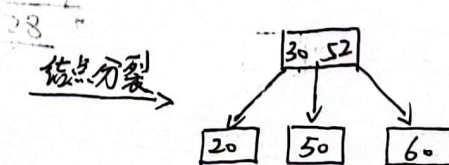
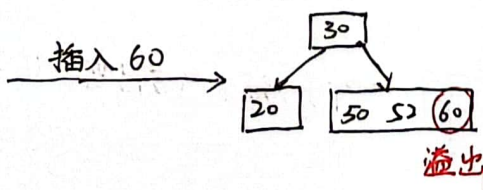
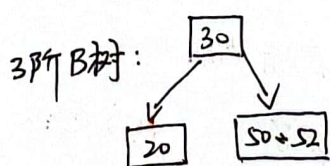
(2) B树的插入

① 通过查找确定插入位置（一定在终端结点）

② 若插入结点关键字个数未超过上限，无需做其他处理。

③ 超过上限，将当前结点的中间元素放到父结点中，当前结点分裂为两个部分。

→ 父结点关键字数 + 1 → 若父结点关键字数超过上限 → 向上分裂（根结点的分裂会导致B树高度+1）



中间位置 $\lceil m/2 \rceil$ 的结点，
↑ 插入原结点的父结点，
(根结点的分裂会导致B树高度+1)

(3) B树的删除

① 非终端结点关键字：用其直接前驱/后继替代其位置，转化为对“终端结点”的删除。

直接前驱：当前关键字左边指针所指子树中“最右下”的元素

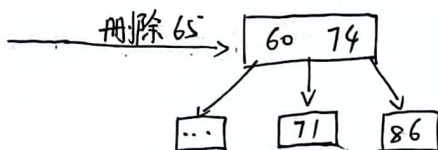
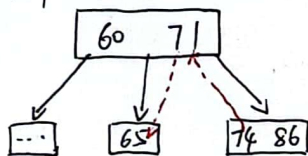
直接后继：当前关键字右边指针所指子树中“最左下”的元素

② 终端结点关键字：删除后关键字个数未低于下限，无需任何处理。

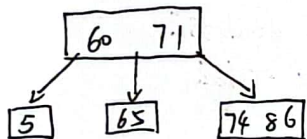
低于下限 { 左兄弟够借，用当前结点的后继、后继的后继依次顶替空缺
右兄弟够借，用当前结点的前驱、前驱的前驱依次顶替空缺
都不够借，则需与父结点内的关键字、左(右)兄弟合并

→ 父结点关键字数量 - 1
可能需继续合并

4阶B树：



兄弟够借



兄弟不够借

5. B+树

(1) 定义：应数据库所需出现的一种B树的变形树。

① 每个分支结点最多有m棵子树(孩子结点)。

② 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。

③ 结点的子树个数与关键字个数相等。

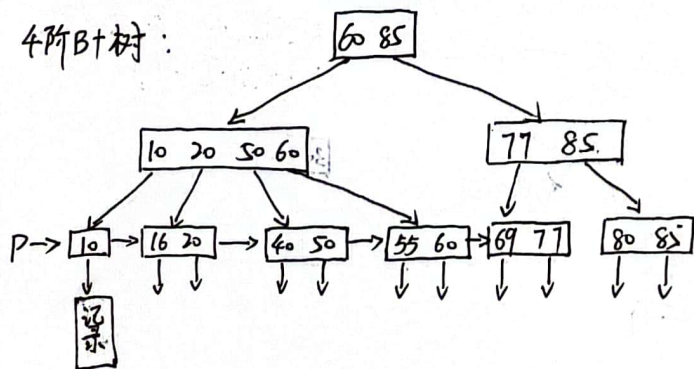
④ 所有叶结点包含全部关键字及指向相应记录的指针，

叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来。

⑤ 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针。

支持顺序查找

4阶B+树：



非叶结点仅起索引作用

— 分支结点

— 叶子结点，包含信息、(记录的存储地址)

可使一个磁盘块存储更多的关键字，树高更矮
使得磁盘读/写次数更少，查找速度更快

查找 { 从根结点开始(多路查找)
从最小关键字开始(顺序查找)

(2) m阶 B+树与m阶 B树的比较

	m阶 B树	m阶 B+树
类比	二叉查找树 $\rightarrow$ m叉查找树	分块查找 $\rightarrow$ 多级分块查找
关键字与分叉	n个关键字对应 $n+1$ 个分叉(子树)	n个关键字对应 n 个分叉(子树)
结点的关键字数	$\begin{cases} \text{根: } 1 \leq n \leq m-1 \\ \text{其他: } \lceil m/2 \rceil - 1 \leq n \leq m-1 \end{cases}$	$\begin{cases} \text{根: } 2 \leq n \leq m \\ \text{其他: } \lceil m/2 \rceil \leq n \leq m \end{cases}$
结点的包含信息	所有结点都包含记录的信息。	只有最下层叶结点包含记录的信息。
关键字是否重复	X	✓
查找方式	- 不支持顺序查找, - 查找成功时, 可能停在任一层 - 查找速度不稳定	- 支持顺序查找 - 成功/失败, 都会停在最下一层 - 查找速度稳定
相同点	- 除根结点外, 最少 $\lceil m/2 \rceil$ 个分叉 - 任何一个结点的子树都要一样高	

四、散列结构 —— 散列表 (Hash 表)

1. 散列表的基本概念.

- (1) 散列表 (哈希表): 一种数据结构, 可以根据数据元素的关键字计算出它在散列表中的存储地址。
- (2) 散列函数: 建立了“关键字” $\rightarrow$ “存储地址”的映射关系。(理想情况下, 查找一个元素时间复杂度 $O(1)$)
- (3) 冲突 (碰撞): 在散列表中插入一个数据元素时, 若插入的位置已经存储了其他元素的情况。
- (4) 同义词: 若不同的关键字通过散列函数映射到同一个存储地址, 则称它们为“同义词”。

2. 散列函数的构造

(1) 注意事项

- ① 定义域必须涵盖所有可能出现的关键字。
- ② 值域不能超出散列表的地址范围。
- ③ 尽可能减少冲突, 散列函数计算出来的地址应尽可能均匀分布在整个地址空间。
- ④ 散列函数应尽量简单

(2) 除留余数法: $H(key) = key \% p$.

① 散列表表长为 m , 取一个不大于 m 但最接近或等于 m 的质数 p .

② 适用场景: 较为通用, 只要关键字是整数即可. 对质数取余, 分布更均匀, 减少冲突.

③ 例子: $key = \{2, 4, 6, 8, 10, 12, 14\}$

$$H(key) = key \% 7$$

14	8	2	10	4	12	6	1
0	1	2	3	4	5	6	7

(3) 直接定址法: $H(key) = key$ 或 $H(key) = a * key + b$.

① a, b 是常数, 计算简单, 且不会发生冲突.

② 适用场景: 关键字分布基本连续.

③ 例子: 用散列表存储班级的 30 个学生信息, 学号为 $\{100025 \sim 100054\}$

$$\text{可令 } H(key) = key - 100025$$

(4) 数字分析法: 选取数码分布较为均匀的若干位作为散列地址

① 设关键字是 r 进制, r 个数码在各位上出现的频率不相同.

② 适用场景: 关键字集合已知, 且关键字的某几个数码位分布均匀.

③ 例子: 电话号码、身份证号码的某几位

(5) 平方取中法: 取 key 的平方值的中间几位作为散列地址

① 得到的散列地址与关键字的每位都有关系.

② 适用场景: 关键字的每位取值都不够均匀.

3. 处理冲突的方法

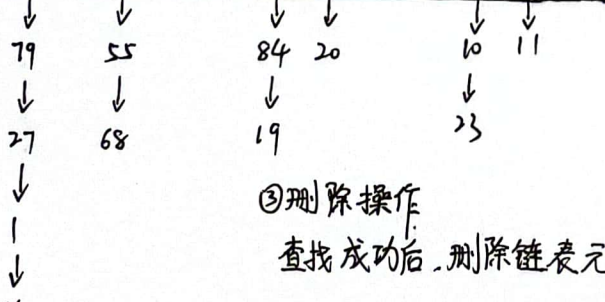
1) 拉链法: 把所有“同义词”存储在一个链表中.

① 插入操作

$key = \{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$

0	1	2	3	4	5	6	7	8	9	10	11	12
1		1		1	1				1	1		1

可采用:
头插法/
尾插法



③ 删除操作

查找成功后, 删除链表元素.

$$H(key) = key \% 13$$

② 查找操作

分析查找长度时, 通常统计“关键字的对比次数”

查找 27, 查找长度 = 2.

20, = 1.

66, 失败, = 4.

21, 失败, = 0.

2) 开放定址法：如果发生“冲突”，就给新元素找另一个空闲位置。

① 线性探测法： $d_i = 0, 1, 2, 3, \dots, m-1$

② 平方探测法： $d_i = 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ ，其中 $k \leq \frac{m}{2}$ 。

③ 双散列法： $d_i = i \times \text{Hash}_2(\text{key})$

若 $\text{Hash}_2(\text{key})$ 计算得到的值与 m 互质，就能保证双散列法可以探测到所有单元。

$H_i = (H(\text{key}) + d_i) \% m$
 初始散列地址 散列表长度
 发生第 i 次冲突时的散列地址 偏移量

④ 伪随机序列： d_i 是一个伪随机序列，如 $d_i = 0, 5, 3, 11, \dots$

注意：删除元素时不能简单地将被删除空间置为空，否则将截断在它之后的探测路径。可以做一个“已删除”标记，进行逻辑删除。

探测覆盖率

线性探测：经过 $m-1$ 次冲突，一定可以探测到所有单元。

平方探测：一般至少可覆盖一半单元；若 m 表示成 $4k+3$ 的素数 (7, 11, ...)，全部单元。

双散列： $\text{Hash}_2(\text{key})$ 与 m 互质，可探测全部单元。

伪随机序列：序列设计合理，可探测全部单元。

4. 散列表查找及性能分析

1) $\text{key} = \{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$ 。

$H(\text{key}) = \text{key} \% 13$ ，采用线性探测处理冲突构造所得散列表如下：

14	1	68	27	55	19	20	84	79	23	11	10						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

查找各关键字的比较次数

关键字: 14	1	68	27	55	19	20	84	79	23	11	10						
次数: 1	2	1	4	3	1	1	3	9	1	1	3						

$$ASL = \frac{1 \times 6 + 2 + 3 \times 3 + 4 + 9}{12} = 2.5$$

2) 查找效率取决于：散列函数、处理冲突的方法、装填因子 a 。

ASL 依赖于 a ，不直接依赖于 n 或 m 。

定义为一个表的装满程度

$$a = \frac{\text{表中记录数 } n}{\text{散列表长度 } m}$$